UFUG 2106 Discrete Mathematics
Written/Programming Assignment 3

Student ID: 50025256 Student Name: 童聪

Remark: There are 6 questions in this assignment. Please record how much time (in minutes) you spent on each question. This information is important to estimate the workload of this assignment and the difficulty of each question. It will NOT affect your grade.

Question	1	2.(1)	2.(2)	3.(1)	3.(2)	4	5	6
Time(min)								

1. Prove that if n is a composite integer, then $n = ab$ where $1 < a \leq \sqrt{n}$ and $b \geq \sqrt{n}$. Use this to show that any composite n must possess a prime factor $p \leq \sqrt{n}$.
(10 marks)

Proof: Let n be a composite integer. Then by definition, $n=ab$ for some integers a,b with $1<a<n$ and $1<b<n$.
Without loss of generality, assume $a \leq b$. Suppose $a > \sqrt{n}$. then $b \geq a > \sqrt{n}$, $n = ab > \sqrt{n} \cdot \sqrt{n} = n$, a contradiction.
Hence, $a \leq \sqrt{n}$. Also, $b = \frac{n}{a} \geq \frac{n}{\sqrt{n}} = \sqrt{n}$ Thus, $n=ab$ with $1<a\leq\sqrt{n}$ and $b \geq \sqrt{n}$
Now, as $a>1$, a has a prime factor p, so $p|a$. Then $p \leq a \leq \sqrt{n}$. By $n=ab$ we have $a|n$. Hence, $p|n$
Therefore, n possess a prime factor $p \leq \sqrt{n}$.

2. A brute-force algorithm known as **trial division** can be used to determine whether a positive integer n is prime or not: we divide n by all primes not exceeding $\sqrt{n}$ and conclude that n is prime if it is not divisible by any of these primes.
   (1) Write the pseudocode of the **trial division** algorithm which tests whether a specified positive integer n is prime or not. Notice that your algorithm can only assume that 2 is a prime. Other primes need to be found by your algorithm.
   (10 marks)
   (2) Given a prime number p, if $2^p - 1$ is also prime, then $2^p - 1$ is called a **Mersenne prime**. Develop a Python program based on your trail division algorithm to find all Mersenne primes up to $2^{40} - 1$. Your program just needs to output the values of p for those Mersenne primes. Name your Python program as **StudentID_2.py**.
   (20 marks)

(1) Algorithm TrialDivision(n)
Input: A positive integer n
Output: True if n is prime, False otherwise

```
if n=1, return False
if n=2, return True
if n mod 2=0, return False
let primes=[2]
i←3
while i*i≤n do
    is_i_prime←True
    for each p in primes do
        if p*p>i, break
        if i mod p=0 then
            is_i_prime←False
            break
```

```
        if is_i_prime = True then
            append i to primes
            if n mod i = 0 then
                return False
        i ← i+2
    return True
```

3. The **Sieve of Eratosthenes** Algorithm is used to find all primes not exceeding a specified positive integer. For instance, the following procedure is used to find the primes not exceeding 100. We begin with the list of all integers between 1 and 100. To begin the sieving process, the integers that are divisible by 2, other than 2, are deleted. Because 3 is the first integer greater than 2 that is left, all those integers divisible by 3, other than 3, are deleted. Because 5 is the next integer left after 3, those integers divisible by 5, other than 5, are deleted. The next integer left is 7, so those integers divisible by 7, other than 7, are deleted. Now we can stop because the next integer left is 11, which is larger than the square root of 100. We conclude that the primes less than 100 are {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}.

   (1) Write the pseudocode of the **Sieve of Eratosthenes** algorithm which can find all the primes not exceeding a specified positive integer n.

   (10 marks)

   (2) Develop a Python program that uses **Sieve of Eratosthenes** algorithm to count the number of prime numbers up to n (denoted by $\pi(n)$), and fill in the following table. For each case, please measure the running time of your Python program in milliseconds. Please also compare the value of $\pi(n)$ with $n/\ln(n)$. What can you observe from the comparison? Name your Python program as **StudentID_3.py**.

n	$\pi(n)$	$n/\ln(n)$	$\frac{\pi(n)}{n/\ln(n)}$	Running time (milliseconds)
100	25	21.7	1.151	0.007
1000	168	144.8	1.161	0.038
10000	1229	1085.7	1.132	0.357
100000	9592	8685.9	1.104	3.891
1000000	78498	72382.4	1.084	50.280
10000000	664579	620420.7	1.071	734.391
100000000	5761455	5428681.0	1.061	8505.477

(20 marks)

Hint: To measure the running time of a piece of Python code, you can refer to the below example. Notice that our problem requires you to output the time in milliseconds.

```
import time
start = time.perf_counter()
# Performing a detailed task, e.g.,
time.sleep(2)
end = time.perf_counter()
elapsed = end - start
print(f"Elapsed Time: {elapsed} seconds")
```

(1) Algorithm SieveOfEratosthenes(n)
Input: A positive integer n
Output: A list of all primes not exceeding n

```
if n<2, return an empty list
let is_prime be an boolean array, indexed from 0 to n
initialize all entries in is_prime to True
set is_prime[0] = False and is_prime[1] = False
p←2
while p×p≤n do
    if is_prime[p] = True then
        for multiple from p×p to n step p do
            is_prime[multiple]←False
    p←p+1
initialize an empty list primes
for i←2 to n do
    if is_prime[i] = True then
        append i to primes

return primes
```

Observation: As n increases, the ratio $\frac{\tau(n)}{n/\ln(n)}$ gets closer and closer to 1

4. The **least common multiple** of the positive integers a and b is the smallest positive integer that is divisible by both a and b. The least common multiple of a and b is denoted by lcm(a, b). Prove that gcd(a, b) · lcm(a, b) = ab. (10 marks)

Proof: Let $d=\gcd(a,b)$, then $a=dm$ and $b=dn$ for some positive integers m,n with $\gcd(m,n)=1$. Since m,n are relatively prime, $\text{lcm}(m,n)=mn$. Hence, $\text{lcm}(a,b)=\text{lcm}(dm,dn)=dmn$. Then $\gcd(a,b)\cdot\text{lcm}(a,b)=d\cdot dmn=d^2mn$ and $ab=dm\cdot dn=d^2mn$. So $\gcd(a,b)\cdot\text{lcm}(a,b)=ab$

5. Use the **extended Euclidean algorithm** to express gcd(2024, 1960) as a linear combination of 2024 and 1960. Show all your steps. (10 marks)

Apply the Euclidean algorithm

$2024=1960\cdot1+64$

$1960=64\cdot30+40$

$64=40\cdot1+24$

$40=24\cdot1+16$

$24=16\cdot1+8$

$16=8\cdot2+0$

so $\gcd(2024,1960)=8$, and

$q_1=1, q_2=30, q_3=1, q_4=1, q_5=1, q_6=2$

Set $s_0=1, s_1=0, t_0=0, t_1=1$ and by $s_j=s_{j-2}-q_{j-1}s_{j-1}$ and $t_j=t_{j-2}-q_{j-1}t_{j-1}$ for $j=2,3,4,5,6$

we have $s_2=1-1\cdot0=1$, $s_3=0-30\cdot1=-30$

$s_4=1-1\cdot(-30)=31$, $s_5=-30-1\cdot31=-61$, $s_6=31-1\cdot(-61)=92$

$t_2=0-1\cdot1=-1$, $t_3=1-30\cdot(-1)=31$, $t_4=-1-1\cdot31=-32$

$t_5=31-1\cdot(-32)=63$, $t_6=-32-1\cdot63=-95$

Hence, $\gcd(2024,1960)=92\cdot2024-95\cdot1960$

6. Use the construction in the proof of the **Chinese remainder theorem** to find all solutions to the system of congruences $x \equiv 1 \pmod 3$, $x \equiv 1 \pmod 5$, $x \equiv 3 \pmod 7$, and $x \equiv 2 \pmod{11}$. Show all your steps. (10 marks)

$m = 3\times5\times7\times11 = 1155$. $M_1 = \frac{m}{3} = 385$, $M_2 = \frac{m}{5} = 231$, $M_3 = \frac{m}{7} = 165$, $M_4 = \frac{m}{11} = 105$. We want to find y_k s.t. $M_k y_k \equiv 1 \pmod{m_k}$

So $385y_1 \equiv 1 \pmod 3 \Rightarrow 1\cdot y_1 \equiv 1 \pmod 3$, so $y_1 = 1$; $231y_2 \equiv 1 \pmod 5 \Rightarrow 1\cdot y_2 \equiv 1 \pmod 5$, so $y_2 = 1$; $165y_3 \equiv 1 \pmod 7 \Rightarrow 4y_3 \equiv 1 \pmod 7$, so $y_3 = 2$; $105y_4 \equiv 1 \pmod{11} \Rightarrow 6y_4 \equiv 1 \pmod{11}$, so $y_4 = 2$. Therefore, $x \equiv 1\cdot385\cdot1 + 1\cdot231\cdot1 + 3\cdot165\cdot2 + 2\cdot105\cdot2 = 2026 \equiv 871 \pmod{1155}$. Hence, all solutions are $x \equiv 871 \pmod{1155}$